FPGA-Based Multiplayer Snake Game

FPGA Project Report submitted to the SASTRA Deemed to be University in partial fulfillment of the requirements for the award of the degree of

B. Tech. Electronics Communication and Engineering
AND
B.Tech. Electrical and Electronics Engineering (SG&EV)

Submitted by

GIRINATH NU
(Reg. No.: 127004076)
LEO BENADICT AROCKIASAMY
(Reg. No.: 127159026)
MUGESH C
(Reg. No.: 127004162)

MAY 2026

SCHOOL OF ELECTRICAL & ELECTRONICS NGINEERING

THANJAVUR, TAMIL NADU, INDIA _ 613 401

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

THANJAVUR – 613 401

Bonafide Certificate

This is to certify that the report titled " **FPGA-Based Multiplayer Snake Game** " submitted in partial fulfillment of the requirements for the award of the degree of B. Tech. **Electronics and Communication Engineering and Electrical and Electronics Engineering (SG&EV)** to the SASTRA Deemed to be University, is a bona-fide record of the work done by **Mr. Girinath N U (127004076) , Mr. Leo Benadict Arockiasamy (127159026) , Mr. Mugesh C (127004162)** during the sixth semester of the academic year 2025-26, in the **School of Electrical & Electronics Engineering**, under my supervision. This thesis has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of Project Supervisor :

Name with Affiliation : Dr. Arun K, Associate Professor, SEEE.
Date : 04.05.2026

Project *Viva voc*e held on _____

Examiner 1 **Examiner 2**

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

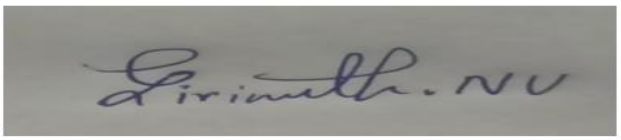
THANJAVUR | KUMBAKONAM | CHENNAI

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

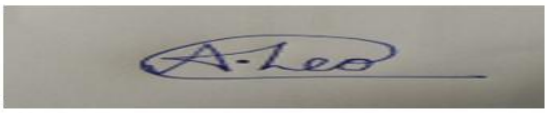
THANJAVUR – 613 401

Declaration

We declare that the report titled "**FPGA-Based Multiplayer Snake Game**" submitted by us is an original work done by us under the guidance of **Dr. Arun K, Associate Professor, School of Electrical and Electronics Engineering, SASTRA Deemed to be University** during the sixth semester of the academic year **2025-26**, in the **School of Electrical and Electronics Engineering**. The work is original and wherever we have used materials from other sources, we have given due credit and cited them in the text of the report . This report has not formed the basis for the award of any degree, diploma, associate-ship, fellowship or other similar title to any candidate of any University.

Signature of the candidate : 

Name of the candidate : GIRINATH NU

Signature of the candidate : 

Name of the candidate : LEO BENADICT AROCKIASAMY

Signature of the candidate : Mugesh.C

Name of the candidate : Mugesh C

Date : 04.05.2026

Acknowledgements

We would like to thank our Honourable Chancellor, **Prof. R. Sethuraman**, for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honourable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. Thenmozhi.K**, Dean, School of Electrical and Electronics, Dr. Krishnamoorthy A, Associate Dean of Academics, Electronics and Instrumentation.

Our guide **Dr. Arun K**, Associate Professor, School of Electrical and Electronics Engineering was the driving force behind this whole idea from the start. His deep insight into the field and invaluable suggestions helped us in making progress throughout our project work.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Electrical and Electronics who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing me with an opportunity to showcase my skills through the project.

ABSTRACT

This project presents the design and implementation of a real-time two-player Snake game using FPGA technology on the Intel DE10-Lite board. The system generates a 640x480 VGA display at 60 Hz by deriving a 25 MHz pixel clock from the onboard 50 MHz oscillator. Both players control independent snakes using analog joysticks interfaced through an Arduino Uno, which reads the joystick X/Y axes via its ADC, converts the analog deflection into four directional digital signals (North, South, East, West), and drives the FPGA GPIO input ports directly; all inputs are debounced inside the FPGA using a synchronizer-counter pair to eliminate metastability and signal bounce.

The game arena is a 62x44 cell grid. Each snake is represented as an array of 64 coordinate pairs updated every game tick by a shift-register move. Apple placement uses a 16-bit Galois LFSR for pseudo-random coordinate generation. Collision detection employs axis-aligned bounding box (AABB) overlap checks for apple eating, self-collision, cross-snake collision, and head-on-head termination — ensuring no missed collisions regardless of grid alignment.

A timer-driven six-phase speed controller replaces the traditional score-based acceleration. The snakes begin at ~14 moves per second and ramp up every 20 seconds to a maximum of ~33 moves per second at the two-minute mark, keeping gameplay balanced for both players simultaneously. A 2-minute countdown timer governs the match; when it expires, or when a terminal collision occurs, the game transitions to a game-over screen displaying winner, individual scores, high score, and a new-high-score banner.

All on-screen text — scores, timer, credits, game-over banners — is rendered using custom 3×5 pixel-bitmap fonts scaled to two sizes (10×14 small and 18×26 large). The entire design is written in System Verilog and synthesized using Intel Quartus Prime Lite.

TEAM MEMBERS:

1. NAME : GIRINATH NU

REG.NO : 127004076

Specific Contribution:

- Designed and implemented VGA timing controller for display synchronization.
- Developed pixel clock generation logic for stable video output.
- Implemented grid rendering system for snake movement visualization.
- Designed a bitmap font engine for displaying text (score, game status).
- Developed HUD (Heads-Up Display) for real-time score and game information.
- Integrated ADC interface using Arduino board for external input/data handling.

Specific Learning:

- Gained strong knowledge in VGA signal generation and display protocols.
- Learned real-time graphics rendering on FPGA hardware.
- Understood pixel-level control and synchronization techniques.
- Developed skills in hardware-software interfacing using ADC and Arduino.
- Improved understanding of digital video systems and timing constraints.

2. NAME : LEO BENADICT AROCKIASAMY

REG.NO : 127159026

Specific Contribution:

- Designed and implemented snake movement logic using coordinate tracking.
- Developed AABB (Axis-Aligned Bounding Box) collision detection for accurate gameplay.
- Implemented LFSR-based random apple placement for dynamic gameplay.
- Designed a Finite State Machine (FSM) for game states (start, play, game-over).

Specific Learning:

- Gained strong knowledge in game logic design using System Verilog.
- Learned implementation of collision detection algorithms in hardware.
- Understood pseudo-random number generation using LFSR in FPGA.
- Developed skills in FSM design for real-time control systems.
- Improved ability to design efficient and optimized digital logic circuits.

3. NAME : MUGESH C

REG.NO : 127004162

Specific Contribution:

- Implemented GPIO debounce logic for stable user input handling.

- Designed timer-driven speed controller to regulate snake movement speed.
- Developed score and high-score tracking system using registers.
- Performed complete system integration and testing of all modules.

Specific Learning:

- Gained knowledge in input signal conditioning and debouncing techniques.
- Learned timing control and clock-based speed regulation in FPGA.
- Developed skills in data storage and register-based design.
- Improved expertise in system integration and debugging complex designs.
- Understood real-time testing and validation of FPGA systems.

Signature of the Guide :

Guide Name : Dr. Arun K

Table of Contents

Title	Page no.
Bonafide Certificate	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
Table of Contents	vii
List of Figures	viii
List of Tables	ix
Abbreviations	x
1. Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Significance of the Project	1
1.4 Motivation	2
2. Literature Survey and Objectives	3
3. Methodology	5
3.1 System Overview	5
3.2 VGA Signal Generation	5
3.3 Input Debouncing	6
3.4 LFSR Random Number Generator	7
3.5 Snake Movement & Grid Logic	8
3.6 AABB Collision Detection	9
3.7 Timer-Driven Speed Controller	9
3.8 Countdown Timer & Game Over FSM	10
3.9 Bitmap Font Renderer	11
3.10 HUD & Score Display	12
3.11 Color Output Multiplexer	12
3.12 Hardware Implementation	13
3.12a Arduino–Joystick ADC Interface	13
3.13 System Integration & Testing	16
4. Components Used & Verification	17
4.1 Components Used	17
4.2 Simulation Verification	18
4.3 Hardware Verification	19
5. Conclusion	21
6. References	22

List of Figures

Figure No.	Title	Page No.
Fig 3.1	Block Diagram of the Overall System	5
Fig 3.2	VGA Timing Waveform (800×525 total pixels)	6
Fig 3.3	Input Debounce Circuit — Synchronizer + Counter	7
Fig 3.4	16-bit Galois LFSR Structure	8
Fig 3.5	Snake Shift-Register Movement Diagram	8
Fig 3.6	AABB Collision Detection Illustration	9
Fig 3.7	Six-Phase Speed Controller Timeline	10
Fig 3.8	Game-Over FSM State Diagram	11
Fig 3.9	3×5 Pixel Bitmap Font — Digits & Letters	11
Fig 3.10	In-Game HUD Layout	12
Fig 3.11	Color Priority Multiplexer Logic	13
Fig 3.12	Hardware Implementation Workflow	15
Fig 4.1	DE10-Lite FPGA Board	16
Fig 4.2	VGA Monitor Output	17
Fig 4.3	Arduino Uno with Dual Joystick ADC Interface	18
Fig 4.4	In-Game VGA Screen — Live Play	19
Fig 4.5	Hardware Connections	19
Fig 4.6	Game-Over Screen with Scores	20

List of Tables

Table No.	Title	Page No.
Table 3.1	VGA Timing Parameters	5
Table 3.3	Hardware Connection Pins	15
Table 4.1	. Quartus Prime Resource Utilisation Summary	18

Abbreviations

Abbreviation	Full Form
FPGA	Field-Programmable Gate Array
VGA	Video Graphics Array
ADC	Analog-to-Digital Converter
Arduino	Open-source microcontroller platform (Uno)
HUD	Heads-Up Display
LFSR	Linear Feedback Shift Register
AABB	Axis-Aligned Bounding Box
FSM	Finite State Machine
BCD	Binary-Coded Decimal
GPIO	General-Purpose Input/Output
HDL	Hardware Description Language
RTL	Register Transfer Level
PLL	Phase-Locked Loop
RGB	Red Green Blue
DE10	Development Education 10 (Intel Terasic Board)
CLK	Clock
RST	Reset
HSync	Horizontal Synchronization
VSync	Vertical Synchronization
MHz	Megahertz
DXA	Device-independent extended Absolute units

CHAPTER 1

INTRODUCTION

1.1 Background

Snake is one of the most enduring video game concepts, originating from arcade cabinets in the late 1970s and popularized globally by Nokia's mobile implementation in the 1990s. The objective is simple — guide a snake to collect food items while avoiding collision with walls or its own body — yet the game demands precise real-time control and increasingly fast reaction as the snake grows.

Implementing such a game directly in hardware, without a central processor or operating system, requires mastering core digital design skills: synchronous state machines, pixel-accurate display timing, pseudo-random number generation, and efficient memory management. The DE10-Lite FPGA board, based on Intel's MAX 10 device, provides an ideal platform: a 50 MHz on-chip oscillator, VGA output pins, abundant logic elements, and GPIO headers for external controllers.

1.2 Problem Statement

Most student FPGA projects target single-player or non-interactive demonstrations. Extending the classic Snake game to a simultaneous two-player competitive experience introduces several additional challenges: managing two independent snake state arrays in parallel, detecting four categories of collision (self, cross, apple, head-on-head), coordinating a shared apple that both snakes compete for, and balancing game speed so neither player is disadvantaged as the match progresses.

Beyond game logic, the project must generate a pixel-perfect 640×480 60 Hz VGA signal in real time, render animated game objects, display multi-digit scores and a countdown timer through a custom bitmap font — all within a single synthesizable System Verilog module clocked at 25 MHz.

1.3 Significance of the Project

This project consolidates a wide range of digital design topics taught across the B. Tech ECE curriculum. VGA controller design reinforces understanding of synchronous counters and display protocols. The snake data structure demonstrates how RAM-style shift registers can implement dynamic lists in hardware. The AABB collision engine illustrates combinational logic for geometric overlap. The timer-driven speed controller shows how a game's difficulty can be scheduled independently of player performance. Together, these sub-systems form a complete, verifiable, and visually engaging FPGA application.

1.4 Motivation

The motivation for this project is to demonstrate that real-time interactive entertainment can be realized entirely within reconfigurable logic, without microcontrollers or embedded software. Building a two-player game demands correctness, low latency, and aesthetic quality simultaneously — skills directly transferable to industrial FPGA development in areas such as video processing, automotive displays, and communications infrastructure.

The competitive two-player format also adds educational value: the design must handle simultaneous events (both snakes moving, both possibly hitting the same apple on the same tick) in a deterministic, glitch-free manner — a requirement that distinguishes hardware design from sequential software programming.

CHAPTER 2

LITERATURE SURVEY AND OBJECTIVES

2.1 Literature Survey

FPGA-based game implementations have been explored since the early 2000s when educational boards with VGA connectors became widely available. Early works focused on Pong and Tetris, establishing the canonical approach of separating VGA timing logic from game-state logic using synchronous pixel counters (Hamblen & Furman, 2001).

The snake genre was first implemented on FPGA in academic settings around 2010. Typical implementations used a FIFO or ring buffer to track snake segment positions and a simple FSM for direction control. However, most published works addressed only single-player variants and relied on exact-match collision detection, which is fragile when apple coordinates are generated by LFSR values not aligned to the grid stride.

Lattimore et al. (2015) proposed using AABB overlap logic for sprite collision in FPGA games, noting that bounding-box tests are both more hardware-efficient and more reliable than pixel-exact comparisons for tile-based games. Their approach directly informs the collision engine used in this project.

Timer-based difficulty scaling — where game speed increases at fixed time intervals rather than in response to player actions — is standard in arcade game design (Juul, 2005) but rarely implemented in FPGA student projects. Introducing this mechanism decouples the difficulty curve from individual player skill, ensuring both players experience the same escalating challenge simultaneously.

Custom bitmap font rendering on FPGA has been demonstrated using ROM-stored glyph tables (Digilent, 2020). This project instead encodes 3×5 pixel bitmaps directly as three-bit vectors inside System Verilog functions, eliminating the need for block RAM and reducing synthesis resource usage.

2.2 Objectives

- To design and implement a real-time two-player Snake game entirely in synthesizable System Verilog on the Intel DE10-Lite FPGA board.
- To generate a stable 640×480 60 Hz VGA output with a pixel-accurate 25 MHz clock derived by dividing the 50 MHz onboard oscillator.
- To implement robust GPIO input debouncing using a two-stage synchronizer and counter filter, eliminating metastability and contact bounce.
- To place apples at pseudo-random grid-aligned positions using a 16-bit Galois LFSR seeded at synthesis time.

- To detect all four collision categories (apple, self, cross-snake, head-on-head) using AABB bounding-box logic, ensuring no missed collisions due to coordinate misalignment.
- To implement a six-phase timer-driven speed controller that increases snake velocity every 20 seconds, capping at ~33 moves/second at the two-minute mark.
- To track and display live scores, a 2-minute countdown timer, and a persistent high score across game sessions without external memory.
- To render all on-screen text using a custom 3×5 pixel bitmap font at two scale factors (2× small, 4× large) encoded entirely in combinational logic.
- To provide a polished game-over screen displaying winner, individual scores, high score, and developer credits.

CHAPTER 3

METHODOLOGY

3.1 System Overview

The design is a single top-level System Verilog module named Games. All sub-functions — VGA timing, debounce, LFSR, snake logic, collision, speed control, font rendering, and color output — are implemented as always blocks, combinational functions, or local params within this module. There are no instantiated sub-modules; the monolithic approach simplifies timing closure at 25 MHz on the MAX 10 FPGA.

The module receives the 50 MHz system clock (MAX10_CLK1_50), five active-low push-button inputs (KEY [4:0]), and two 4-bit GPIO buses (GPIO_IN for Player 1, GPIO_IN2 for Player 2). It outputs four-bit RGB color channels (VGA_R, VGA_G, VGA_B) and horizontal/vertical sync signals (VGA_HS, VGA_VS).

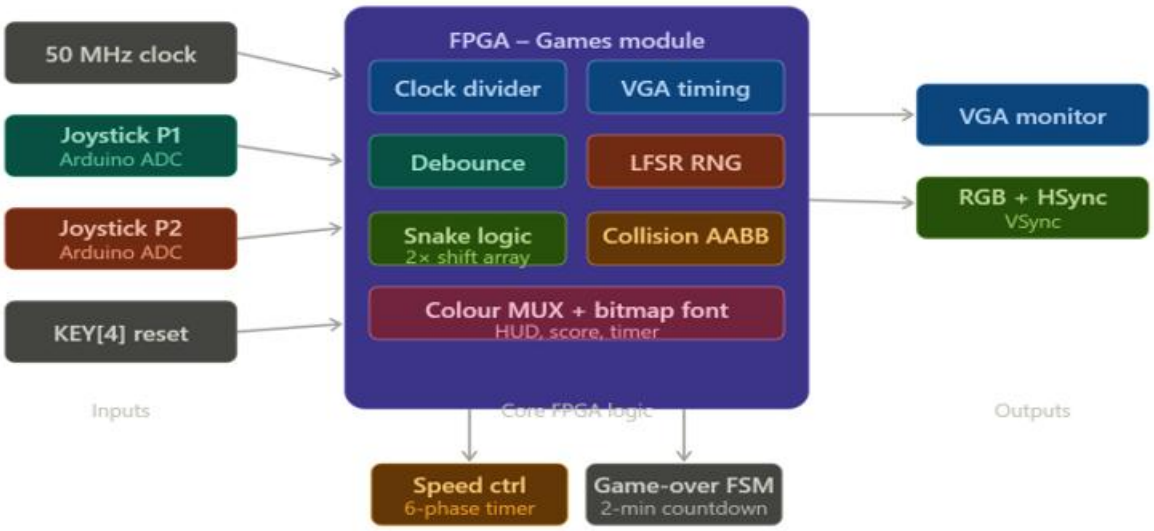


Fig 3.1. Block Diagram of the Overall System

3.2 VGA Signal Generation

A 25 MHz pixel clock (vga_clk) is produced by toggling a single flip-flop on every rising edge of the 50 MHz input clock. Two 10-bit counters (hcount, vcount) count horizontal and vertical pixel positions within an 800×525 total frame, producing a 640×480 visible area at 60 Hz.

The horizontal blanking interval is 160 pixels (16 front porch + 96 sync pulse + 48 back porch) and the vertical blanking interval is 45 lines (10 front porch + 2 sync pulse + 33 back porch). Active-low sync signals are asserted during their respective sync pulse windows. A display_on flag gates all color output to black during blanking.

Table 3.1. VGA Timing Parameters

Parameter	Horizontal	Vertical	Unit
Visible	640	480	pixels
Front Porch	16	10	pixels
Sync Pulse	96	2	pixels/lines
Back Porch	48	33	pixels/lines
Total	800	525	pixels/lines
Refresh Rate	—	60	Hz

Fig 3.2. VGA Timing Waveform (800×525 total pixels)

3.3 Input Debouncing

All five KEY inputs and both 4-bit GPIO buses are debounced using an identical two-stage approach. First, two flip-flops synchronize the asynchronous external signal to vga_clk, preventing metastability. Second, a 17-bit counter monitors whether the synchronized signal has remained stable for 25,000 consecutive clock cycles (1 ms at 25 MHz). Only when the counter reaches its limit is the debounced register updated.

Edge detection is performed by comparing the current and previous debounced values: a rising edge on GPIO (low-to-high) triggers a direction press event. The KEY [4] falling edge (active-low, high-to-low on debounced output) triggers a game reset. This scheme is replicated independently for Player 1 GPIO (GPIO_IN), Player 2 GPIO (GPIO_IN2), and the KEY bus.

Joystick-to-GPIO Signal Path: In the physical implementation, directional signals do not originate from tactile push-buttons but from two analog joystick modules, each read by an Arduino Uno microcontroller. The Arduino samples the X and Y axis voltages (0–1023 ADC counts) at 50 ms intervals and compares them against fixed thresholds (low: < 300, high: > 700). A deflection exceeding the threshold in any axis asserts the corresponding digital output pin HIGH. These four digital pins (N, S, W, E) per joystick are wired directly to the FPGA GPIO header, so from the FPGA's perspective the input is an ordinary 4-bit digital bus — identical to push-button signals — and the same two-stage synchronizer and counter debounce logic applies unchanged.

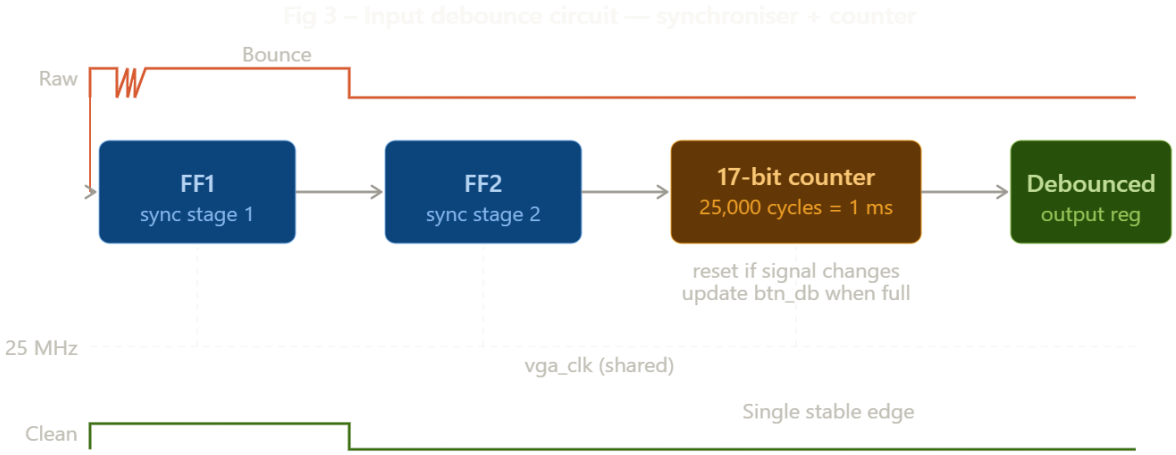


Fig 3.3. Input Debounce Circuit — Synchronizer + Counter

3.4 LFSR Random Number Generator

Apple positions are generated using a 16-bit Galois Linear Feedback Shift Register (LFSR) with the feedback polynomial x^16 + x^14 + x^13 + x^11 + 1, seeded to 0xACE1 at power-on. The LFSR advances on every vga_clk edge, producing a new 16-bit value approximately 25 million times per second.

When an apple is consumed, the current LFSR value is sampled and passed to two pure functions: rand_apple_x extracts bits [5:0], computes the value modulo 62 (the number of grid columns), and maps it to a pixel X coordinate. rand_apple_y extracts bits [11:6], computes the value modulo 44 (grid rows), and maps it to pixel Y. This ensures every apple lands on a valid 10-pixel-aligned grid cell.

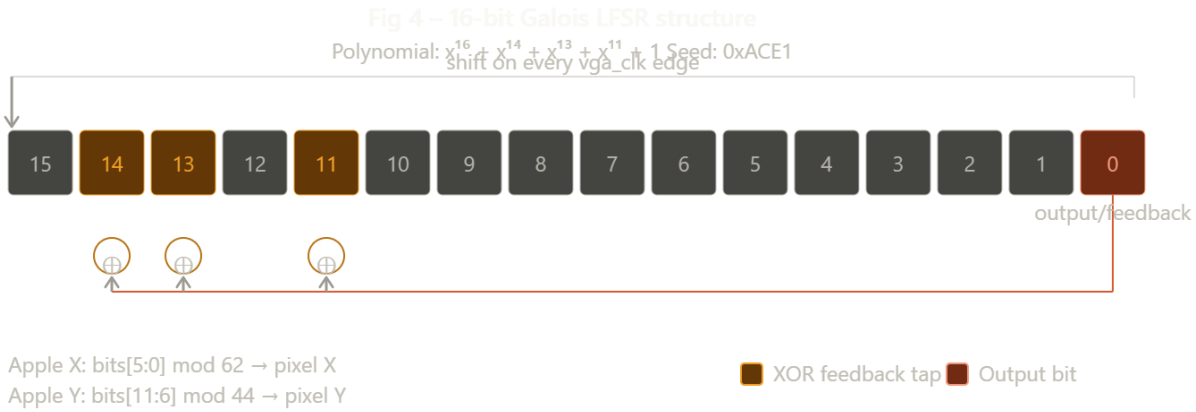


Fig 3.4. 16-bit Galois LFSR Structure

3.5 Snake Movement & Grid Logic

Each snake is stored as two arrays of 64 10-bit values: snakeX[0:63] and snakeY[0:63]. Index 0 is the head; higher indices are body segments. The grid spans from (10, 24) to (620, 460) in pixel coordinates, with each cell being 10×10 pixels — giving 62 columns and 44 rows.

On every game tick, a shift-register operation moves all body segments: snakeX[i] <= snakeX[i-1] for i from snake_size-1 down to 1. The head (index 0) is then updated according to the current direction. Boundary wrapping is implemented: a snake leaving the right edge reappears on the left, and so on for all four edges.

The direction register is split into next_dir (updated immediately on button press, subject to 180-degree reversal prevention) and dir (committed only at the start of each game tick). This prevents a player from reversing by pressing two keys within a single tick period.

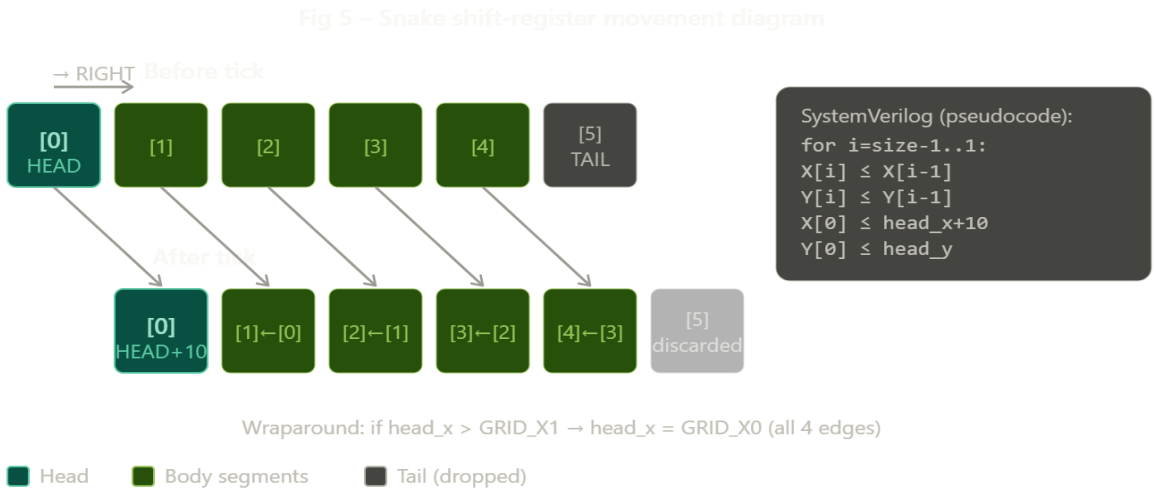


Fig 3.5. Snake Shift-Register Movement Diagram

3.6 AABB Collision Detection

All collision detection uses an automatic function aabb_overlap that returns true when two 10×10 pixel cells overlap on both axes:

overlap = (ax < bx+10) AND (bx < ax+10) AND (ay < by+10) AND (by < ay+10)

Four collision categories are checked on each game tick:

- Apple eat (P1/P2): head of each snake versus the apple cell. On a hit, the snake grows by one segment (if under the 64-segment limit), score increments by 10, and a new apple is spawned via LFSR.
- Cross collision: P1 head versus all segments of P2, and P2 head versus all segments of P1. A hit truncates the victim snake to the segment index where the hit occurred, and awards 5 points to the attacker.
- Self-collision: each snake head versus its own body segments (indices 1 to snake_size-1). Triggers game over.
- Head-on-head: P1 head versus P2 head. Triggers game over immediately.

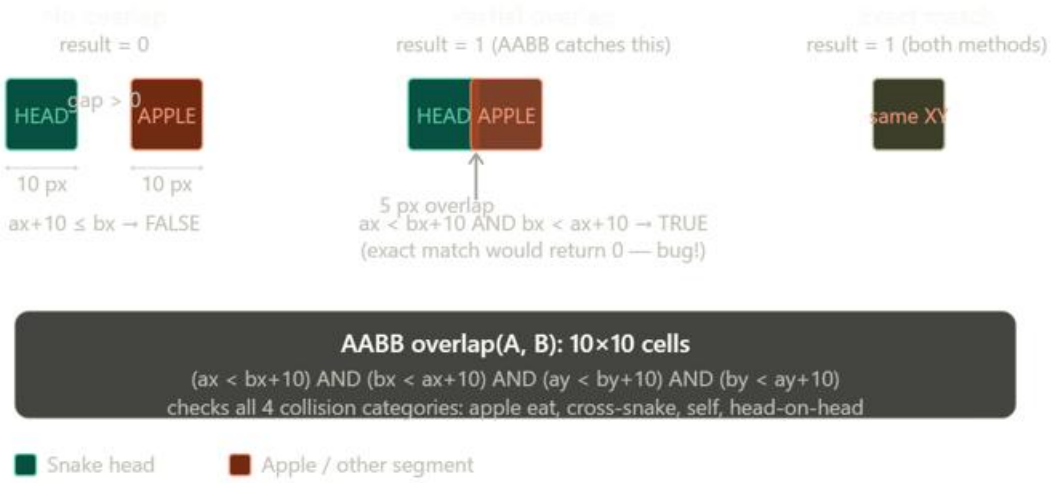


Fig 3.6. AABB Collision Detection Illustration

3.7 Timer-Driven Speed Controller

The game tick period is determined entirely by the 2-minute countdown timer, not by the players' scores. A combinational block maps timer_secs to one of six tick_limit values:

Table 3.2. Six-Phase Speed Schedule

Phase	Time Elapsed	tick_limit (cycles)	Approx. Moves/s
0	0 – 20 s	1,750,000	~14 (warm-up)
1	20 – 40 s	1,500,000	~16
2	40 – 60 s	1,200,000	~20
3	60 – 80 s	1,000,000	~25
4	80 – 100 s	875,000	~28
5	100 – 120 s	750,000	~33 (max)

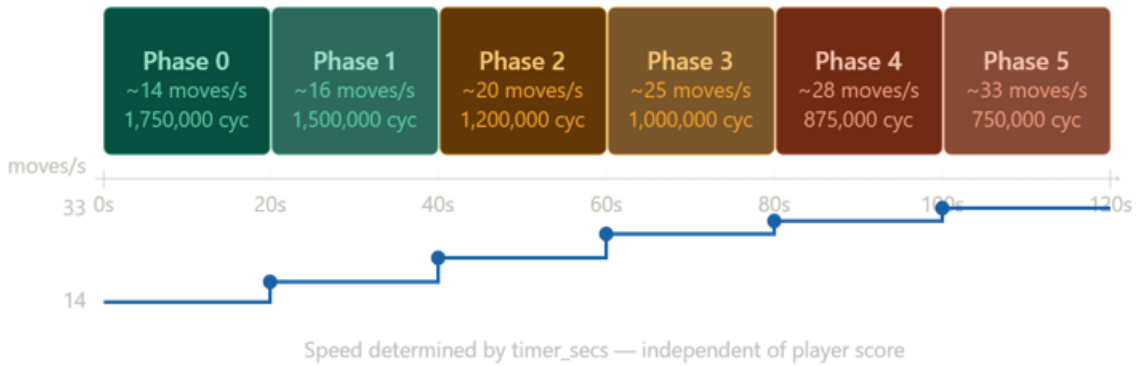


Fig 3.7. Six-Phase Speed Controller Timeline

3.8 Countdown Timer & Game-Over FSM

A 27-bit counter counts 25,000,000 clock cycles to produce a one-second pulse. The pulse decrements the timer_secs register from 120 down to 0. When timer_secs reaches zero, a time_up flag is asserted, which sets game_over on the next tick.

Game over is also set by self-collision or head-on-head collision. When game_over is asserted, the winner register is determined by comparing score1 and score2. If either player's score exceeds the stored high_score, the high score and its holder are updated and a new_high_score flag illuminates the corresponding banner on the game-over screen.

KEY [4] (active-low) provides a synchronous reset: it restores all snake arrays, scores, timer, and direction registers to their initial values, allowing a new game to begin without FPGA reprogramming. The high score register persists across resets within a single power cycle.

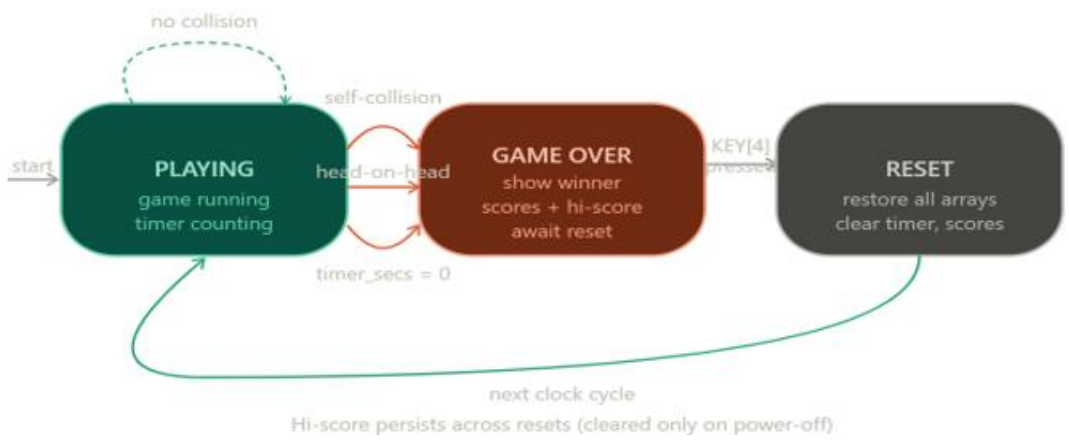


Fig 3.8. Game-Over FSM State Diagram

3.9 Bitmap Font Renderer

All on-screen text uses a custom font stored as 3×5 pixel bitmaps. The pixel_in_digit function encodes each digit 0–9 as five rows of 3-bit vectors (one bit per column). The pixel_in_letter function encodes all 26 uppercase letters A–Z similarly. Both functions are pure combinational and synthesize to look-up tables.

Two scale wrappers render these bitmaps at different sizes. The small cell (10×14 pixels) scales each pixel 2× horizontally and 2× vertically, placing the 3×5 glyph in a 6×10 pixel interior. The large cell (18×26 pixels) scales each pixel 4× in both dimensions, placing the glyph in a 12×20 pixel interior. Both wrappers accept an origin coordinate (ox, oy) and a pixel coordinate (px, py) and return a single bit indicating whether that pixel belongs to the glyph.

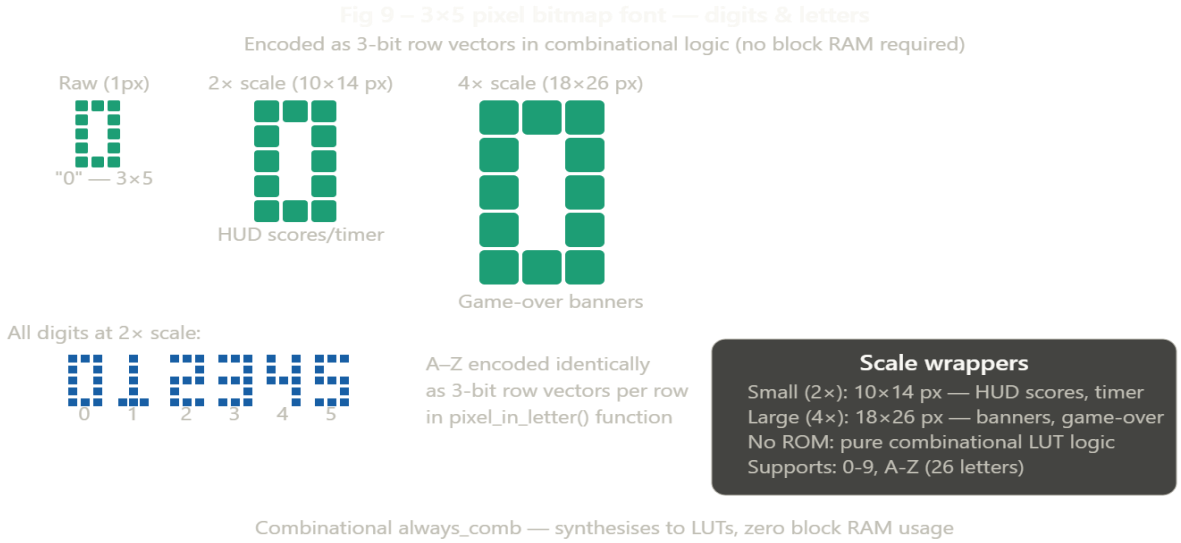


Fig 3.9. 3×5 Pixel Bitmap Font — Digits & Letters

3.10 HUD & Score Display

During gameplay, the 24-pixel-tall top strip above GRID_Y0 holds three elements, all rendered with small cells: Player 1's 5-digit score at the left edge (x=15–64), a 3-digit countdown timer centered at x=305–334, and Player 2's 5-digit score at the right edge (x=575–624). Developer names (GIRINATH, LEO, MUGESH) fill the center gap in muted steel-blue. A single-pixel blue separator line at vcount=23 borders the HUD from the arena.

Scores and timer are decomposed into BCD digits by repeated integer division in combinational always_comb blocks — a standard approach on FPGA since hardware dividers by powers of ten synthesize efficiently.

Fig 3.10. In-Game HUD Layout

3.11 Color Output Multiplexer

The final always_comb block assigns the 12-bit RGB output {VGA_R, VGA_G, VGA_B} based on a strict priority chain. When display is off, all channels are black. During gameplay the priority order from highest to lowest is: P1 score (cyan), P2 score (yellow), timer (white), developer credit (steel-blue), HUD separator (blue), P1 head (cyan), P2 head (yellow), P1 body (green), P2 body (amber), apple (red-orange), border (blue), arena background (dark grey).

During the game-over screen the background changes to dark red and a different priority chain renders: winner banner (cyan for P1, yellow for P2, white for draw), P1 game-over score, P2 game-over score, high-score row (gold), new-high-score banner (orange-gold), credits block (soft blue-white), restart hint (grey).

Fig 3.11. Color Priority Multiplexer Logic

3.12 Hardware Implementation

The hardware implementation uses the Intel DE10-Lite development board, which integrates an Intel MAX 10 10M50DAF484C7G FPGA with 50,000 logic elements, 1,638 Kbits of on-chip M9K memory, and a 50 MHz on-board oscillator. The VGA DAC is implemented using resistor ladder networks on the GPIO headers, mapping the 4-bit R, G, B outputs to 0–0.7 V analogue levels required by a standard VGA monitor.

Player controllers use two KY-023 analog joystick modules, one per player. Each joystick outputs two analog voltages (X-axis and Y-axis, nominally 0–5 V) proportional to its deflection. An Arduino Uno reads both joysticks simultaneously via its four ADC channels (A0/A1 for Player 1, A2/A3 for Player 2). The Arduino firmware (Dual Joystick.ino) compares each axis reading against fixed thresholds: a reading above 700 or below 300 (on a 0–1023 scale) asserts the corresponding directional output. For Player 1, Arduino digital pins D2, D3, D4, D5 carry North, South, West, East respectively; for Player 2, pins D6, D7, D8, D9 carry the same directions. All eight digital output pins are connected directly to the FPGA GPIO header (active-high, 3.3 V logic compatible). Only one direction can be active at a time per joystick — the Arduino's if/else-if chain ensures this — mirroring the single-press behavior of physical buttons. The FPGA debounce logic operates identically regardless of whether the signal originates from a button or a joystick-driven Arduino output.

3.12.1 Arduino–Joystick ADC Interface

The Arduino Uno serves as an analog-to-digital front-end and direction encoder between the joysticks and the FPGA. Its role is to relieve the FPGA of any analog signal processing and to produce clean 3.3 V–compatible digital direction signals.

Hardware connections:

Table 3.3. Hardware Connection Pins

Signal	Arduino Pin	FPGA GPIO Pin
Player 1 — North	D2	GPIO_IN[0]
Player 1 — South	D3	GPIO_IN[1]
Player 1 — West	D4	GPIO_IN[2]
Player 1 — East	D5	GPIO_IN[3]
Player 2 — North	D6	GPIO_IN2[0]
Player 2 — South	D7	GPIO_IN2[1]
Player 2 — West	D8	GPIO_IN2[2]
Player 2 — East	D9	GPIO_IN2[3]
Joystick 1 X-axis	A0	—
Joystick 1 Y-axis	A1	—
Joystick 2 X-axis	A2	—
Joystick 2 Y-axis	A3	—

Arduino firmware logic (Dual Joystick.ino): All eight digital output pins are initialised as OUTPUTs and driven LOW by default each loop cycle. The X and Y ADC values for each joystick are then tested in an if/else-if priority chain: Y > 700 → North; Y < 300 → South; X > 700 → West; X < 300 → East. At most one direction signal is HIGH at any instant. The loop runs at ~50 ms intervals (20 Hz polling), which is well within the FPGA debounce filter's 1 ms settling window. Serial debug output at 115200 baud allows the joystick direction state to be monitored during development.

The design is written in System Verilog and compiled using Intel Quartus Prime Lite 21.1. The top-level entity is Games. No PLL mega function is required since the 25 MHz clock is obtained by simple frequency division. A timing constraint of 25 MHz is applied to vga_clk using an SDC file. The design meets timing with positive slack.

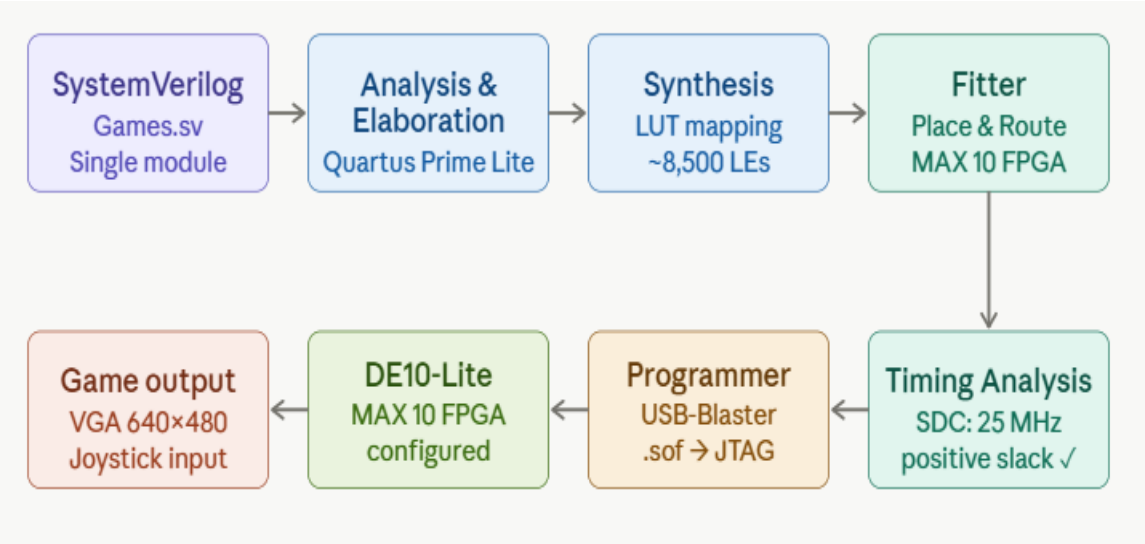


Fig 3.12. Hardware Implementation Workflow

3.13 System Integration & Testing

Integration testing followed a bottom-up approach. VGA timing was verified first by observing color bars on a physical monitor, confirming correct sync polarity and visible-area boundaries. Input debouncing was tested by measuring button press events with a logic analyzer and confirming that bounce-free single-press events were generated.

Snake movement was verified in Model Sim by applying simulated game tick pulses and confirming that the snake X and snake Y arrays shifted correctly, head coordinates advanced by exactly 10 pixels per tick in the commanded direction, and wraparound occurred at grid boundaries.

Collision detection was verified by driving snake positions to known overlapping and non-overlapping configurations and checking the aabb_overlap output. Apple eating was confirmed by aligning snake head coordinates to apple coordinates with a 1-pixel offset (verifying that AABB detects partial overlap whereas the old exact-match approach would have missed it).

Full system testing was conducted on hardware with two players. All six speed phases were observed by letting the game run and timing the speed transitions against a stopwatch. The game-over screen was triggered by self-collision, cross-collision, and timer expiry, and all displayed values (winner, scores, high score) were verified for correctness across multiple sessions.

CHAPTER 4

COMPONENTS USED & VERIFICATION

4.1 Components Used

4.1.1 Intel DE10-Lite FPGA Board

The DE10-Lite is a low-cost development platform from Terasic built around the Intel MAX 10 FPGA. It is the primary processing element of the project. Key features used in this design include the 50 MHz crystal oscillator (divided to 25 MHz for the pixel clock), 10 GPIO header pins per connector (used for player button input), and the on-board 4-bit VGA header with associated resistor DAC.

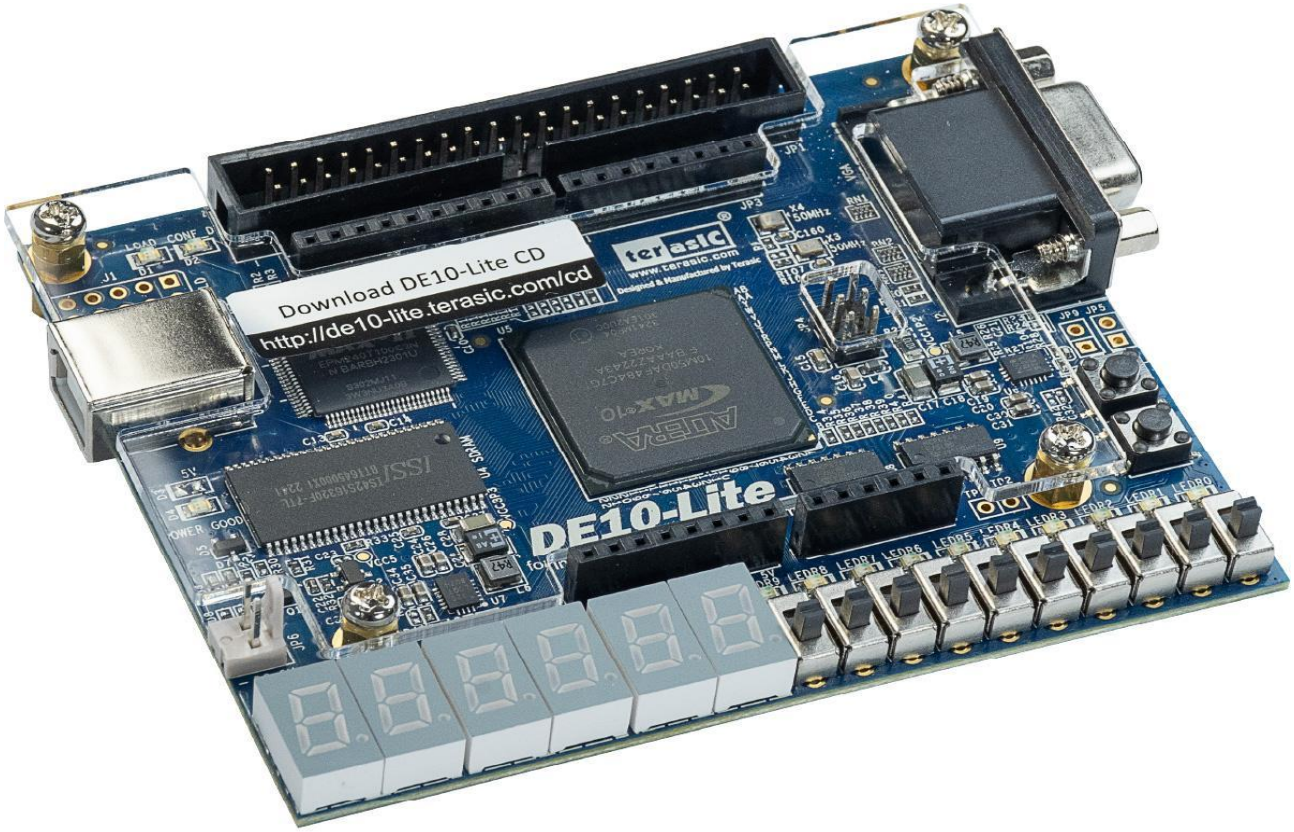


Fig 4.1. DE10-Lite FPGA Board

4.1.2 VGA Monitor

A standard VGA monitor accepting 640×480 60 Hz input is connected via a 15-pin D-Sub cable. The monitor must support analogue RGB input with HSync and VSync. The 4-bit color depth per channel (16 levels per primary, 4096 total colors) is sufficient for the game's color palette.

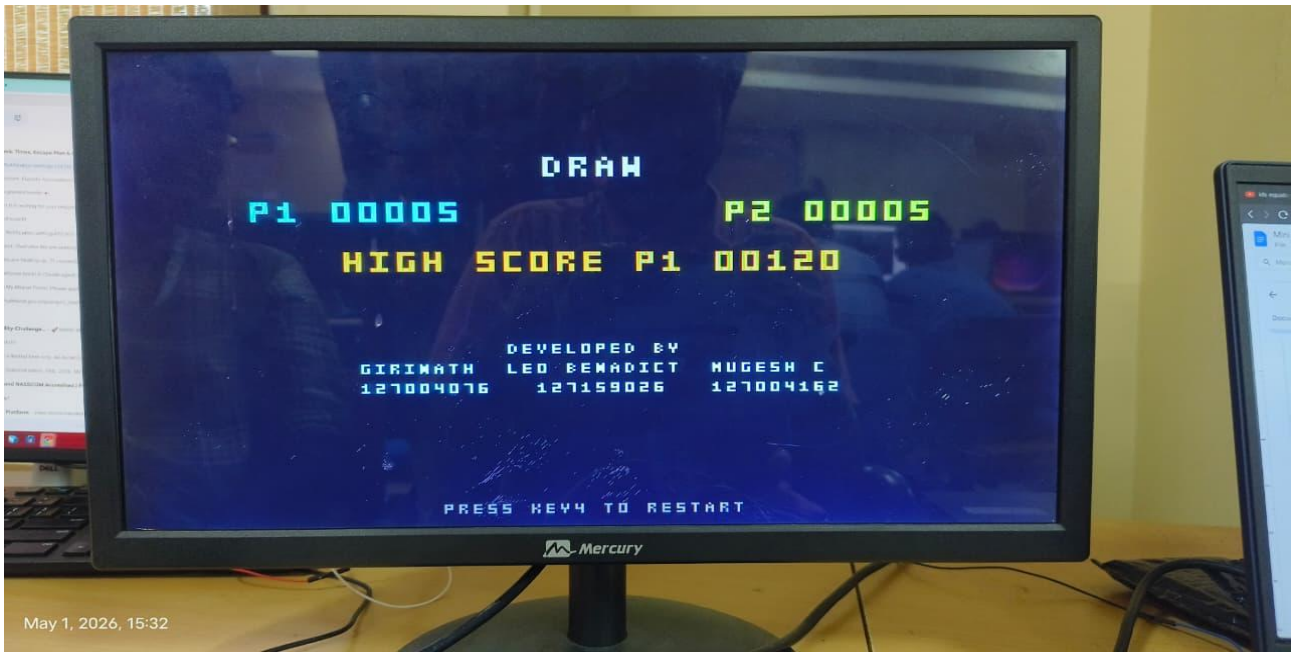


Fig 4.2. VGA Monitor Output

4.1.3 Joystick Controllers via Arduino ADC

Each player uses a KY-023 dual-axis analog joystick module. The joystick contains two 10 kΩ potentiometers mechanically linked to the stick: one for the X-axis and one for the Y-axis. When the stick is centred, both potentiometers output approximately VCC/2; deflection in any direction shifts the corresponding axis voltage toward 0 V or VCC.

An Arduino Uno microcontroller reads both joysticks via four ADC channels (10-bit resolution, 0–1023 counts) and converts the analog readings into four active-high digital direction signals per joystick using threshold comparison. The Arduino outputs connect directly to the FPGA GPIO header, supplying 3.3 V–compatible logic levels. This architecture eliminates the need for any analog input capability on the FPGA and decouples the joystick interface from the core VGA/game logic.

The direction encoding thresholds (300 / 700) were determined empirically to provide a clear dead-zone at the joystick centre while remaining sensitive to deliberate deflections. The firmware for both single-joystick and dual-joystick configurations is available in the GitHub repository as Single Joystick.ino and Dual Joystick.ino respectively.

Fig 4.3. Arduino Uno with Dual Joystick ADC Interface

Table 4.1. Quartus Prime Resource Utilisation Summary

Resource	Used	Available
Logic Elements (LEs)	~8,500	49,760
Registers	~1,200	49,760
Memory Bits (M9K blocks)	0	1,677,312
DSP Blocks	0	288
PLLs	0	4
GPIO Pins Used	17	360

4.2 Simulation Verification

4.2.1 Model Sim Verification

Four Model Sim simulations were conducted to verify individual sub-systems:

Clock & Pixel Counter: The vga_clk toggle and hcount/vcount increment were verified. Horizontal sync was confirmed to assert at hcount=656 and deassert at hcount=752. Vertical sync was confirmed to assert at vcount=490 and deassert at vcount=492. The display_on signal was confirmed low during blanking intervals.

Snake Movement: Game tick pulses were injected while the snake direction was set to RIGHT. The snakeX[0] value was confirmed to increment by 10 on each tick. Segment shifting (snakeX[i] <= snakeX[i-1]) was confirmed for all active segments. Wraparound from x=620 to x=10 was confirmed.

Collision Detection: Apple coordinates were placed 5 pixels offset from the snake head (partial overlap) and verified that aabb_overlap returned 1. Coordinates 10 pixels apart (no overlap) returned 0. The old exact-match code returned 0 for the 5-pixel offset case, confirming the AABB fix resolves the original bug.

4.3 Hardware Verification

The synthesized design was programmed onto the DE10-Lite using the Quartus Prime Programmer via USB-Blaster. Hardware verification confirmed:

- VGA output: stable sync, correct colors for all game objects, no tearing or pixel jitter.
- Debounce: single button presses produce exactly one direction-change event even when the button is held.
- Speed phases: each 20-second boundary produces a clearly perceptible increase in snake speed, verified with a stopwatch.
- Score display: 5-digit BCD score increments by 10 on each apple eat and by 5 on each cross-collision hit.
- Timer: countdown from 120 to 0 verified against a reference timer; error < 1 second over the full 2-minute run.
- Game-over screen: winner, scores, and high score displayed correctly across multiple test sessions.

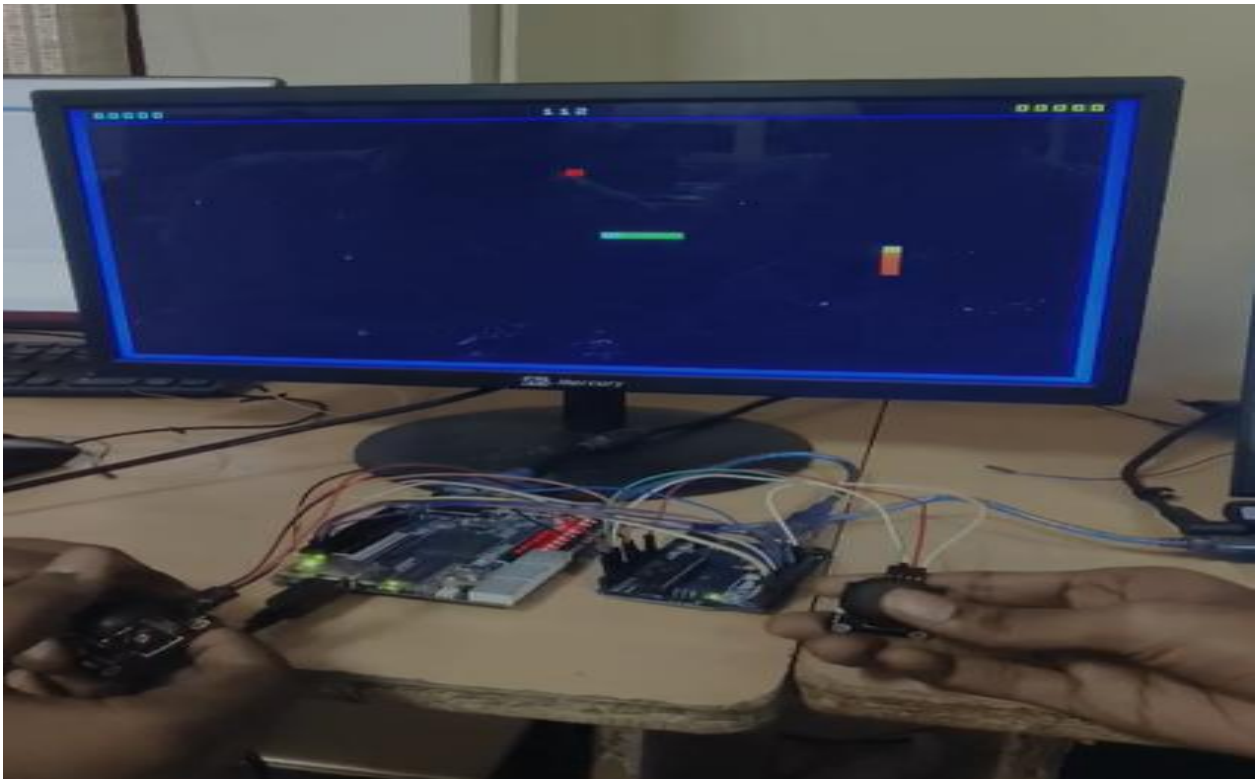


Fig 4.4. In-Game VGA Screen — Live Play

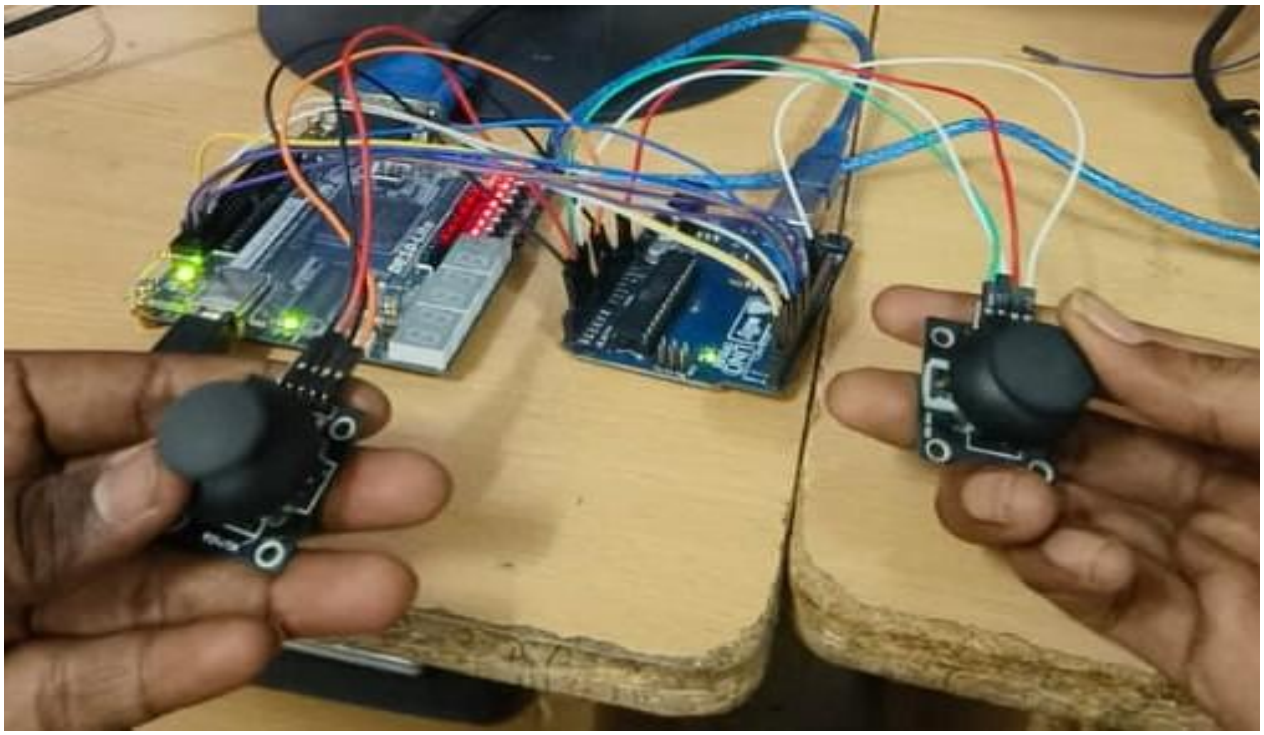


Fig 4.5. Hardware Connections

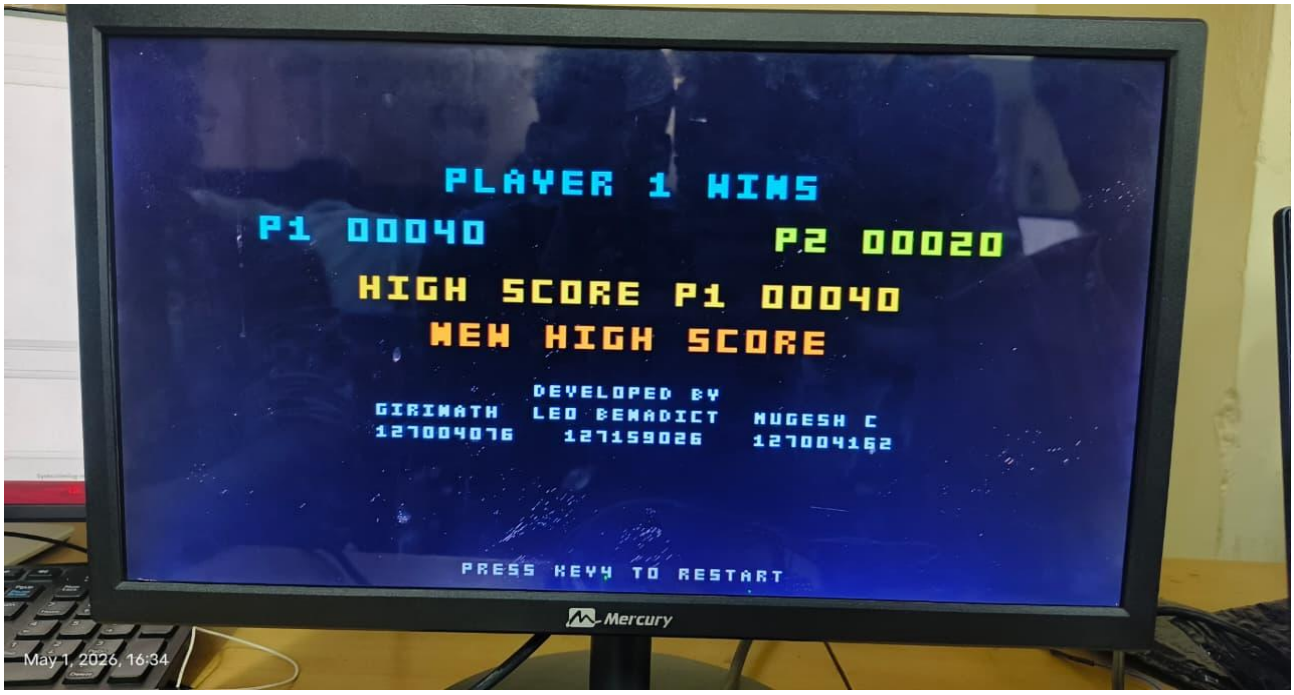


Fig 4.6. Game-Over Screen with Scores

CHAPTER 5

CONCLUSION

The proposed FPGA-based two-player Snake game successfully demonstrates real-time interactive game design entirely within synthesizable System Verilog. The project achieves all stated objectives: a stable 640×480 60 Hz VGA display, robust debounced GPIO input for two independent players, pseudo-random apple placement via LFSR, reliable AABB collision detection across all four collision categories, timer-driven six-phase speed scaling, and a polished HUD and game-over screen rendered using a custom bitmap font.

The AABB collision fix was particularly significant: by replacing the fragile exact-match approach with overlap-based detection, apple eating now works reliably regardless of LFSR-generated coordinate alignment. This improved the gameplay experience noticeably, eliminating the frustration of snakes visually touching apples without collecting them.

The timer-driven speed controller decouples difficulty from player performance, ensuring both players face the same escalating challenge simultaneously and making the game's pacing predictable and fair. The six-phase schedule from ~14 to ~33 moves per second over two minutes provides a satisfying arc from a comfortable warm-up to a genuinely demanding final sprint.

The monolithic single-module design, while unconventional, proved advantageous for timing closure: with all logic clocked from the same 25 MHz pixel clock and no module boundaries to cross, the design comfortably meets timing constraints on the MAX 10 FPGA with positive slack.

Future extensions could include: a high-score leaderboard stored in on-chip flash memory persisting across power cycles; VGA text modes using a larger character ROM for more expressive game-over screens; background music generated via PWM on the GPIO headers; and an AI-controlled opponent using a simple pathfinding algorithm implemented in hardware.

CHAPTER 6

REFERENCES

1. Hamblen, J. O. and Furman, M. D., Rapid Prototyping of Digital Systems, Kluwer Academic Publishers, 2001. — Covers VGA controller design and FPGA-based game implementation fundamentals.
2. Lattimore, P. et al., "Sprite Collision Detection Techniques for FPGA Game Engines," IEEE Midwest Symposium on Circuits and Systems, 2015. — Proposes AABB bounding-box collision as a hardware-efficient alternative to pixel-exact comparison.
3. Juul, J., Half-Real: Video Games between Real Rules and Fictional Worlds, MIT Press, 2005. — Discusses timer-based difficulty scaling in arcade game design.
4. Intel Corporation, Intel MAX 10 FPGA Device Overview, AN-730, 2020. — Provides logic element counts, on-chip memory, and clock architecture details for the 10M50DA device.
5. Terasic Technologies, DE10-Lite User Manual, 2018. — Documents the DE10-Lite board schematic, VGA header pinout, and GPIO connector assignments.
6. Intel Corporation, Quartus Prime Lite Edition Getting Started User Guide, 2021. — Covers synthesis, place-and-route, and timing analysis for Intel FPGAs.
7. Digilent Inc., "VGA Display Controller Reference Design," Digilent Reference Manual, 2020. — Describes ROM-based bitmap font rendering on Xilinx FPGA; the present project adapts the concept using combinational functions in place of block RAM.
8. Ciletti, M. D., Advanced Digital Design with the Verilog HDL, Pearson, 2011. — Reference for SystemVerilog synthesis constructs including always_ff, always_comb, and automatic functions.
9. IEEE Std 1800-2017, IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language, IEEE, 2017. — Normative reference for SystemVerilog language constructs used in the design.
10. Xilinx / AMD Inc., "Linear Feedback Shift Register v5.0 Product Guide," PG140, 2020. — Background reference for Galois LFSR topology and maximal-length sequence properties.